

Algorithm2: *Problem Solving* with *Algorithms* and *Data Structures*

Lecture 02.

무식하게 풀기

경북대학교 배준현 교수
(joonion@knu.ac.kr)



무식하게 풀기: *Brute-Force* Approach

가능한 경우의 수를 일일이 나열하면서 답을 찾는 방법

예) 두 정점 사이의 최단 경로를 찾는 문제

1. 두 정점 사이의 모든 경로들을 하나 하나 전부 만들어서,
2. 그 중에서 가장 짧은 경로를 찾는다.

어떤 문제를 풀기 위한 가장 단순하고 가장 확실한 방법:

새로운 문제를 마주하면 가장 먼저 질문한다: 무식하게 풀 수 있을까?



완전 탐색: *Exhaustive Search*

가능한 솔루션 공간 전체를 일일이 모두 탐색해 보는 알고리즘

문제의 특성에 대한 직관적인 접근이 가능하고 가장 구현하기 쉬운 방법

예) 열 명의 학생을 한 줄로 세우되, 사이가 안 좋은 학생을 떨어뜨리는 방법

1. 열 명의 학생을 한 줄로 세우는 모든 방법을 탐색
탐색해야 할 경우의 수: $10! = 3,628,820$
2. 사이가 안 좋은 학생들이 붙어있는 경우의 수를 제외



재귀: *Recursion*

재귀 함수란 자신이 수행할 작업을 유사한 형태의 여러 조각으로 쪼개 뒤,
그 중 한 조각을 수행하고, 나머지를 자기 자신을 호출해 실행하는 함수

종료 조건: *exit condition*

기저 사례(*base case*): 더이상 쪼개지지 않는 최소한의 작업(답을 반환)

재귀 조건: *recursive condition*

더 작은 부분 문제로 쪼개어 재귀적으로 문제를 해결



재귀로 무식하게 풀기

재귀 호출을 이용하면 특정 조건을 만족하는 조합(combination)을 모두 생성하는 코드를 쉽게 작성할 수 있다. (DFS, Backtracking)

따라서, 재귀 호출은 완전 탐색을 구현할 때 아주 유용한 도구이다.



재귀 .vs. 반복: *recursion* .vs. *iteration*

문제: 0부터 $n - 1$ 까지의 정수 중 4개를 고르는 모든 경우의 수를 출력하시오.

```
def icomb(n):  
    for a in range(n):  
        for b in range(a + 1, n):  
            for c in range(b + 1, n):  
                for d in range(c + 1, n):  
                    print(a, b, c, d)
```

```
def rcomb(n, comb=[], start=0):  
    if len(comb) == 4:  
        print(*comb)  
    else:  
        for i in range(start, n):  
            rcomb(n, comb + [i], i + 1)
```



탐색 공간과 상태 공간 트리

탐색 공간: *search space*

문제에서 고려할 수 있는 모든 가능한 해(solution)의 집합

상태 공간 트리: *state space tree* (SST)

문제의 탐색 공간을 탐색하는 과정에서 발생할 수 있는 모든 상태를 트리 형태로 표현한 것

부분 해는 트리의 각 노드로 표현하고, 각 가지(*branch*)는 가능한 선택을 나타낸다.

루트 노드가 초기 상태가 되고, 리프 노드가 모든 선택이 끝난 최종 해답의 후보가 된다.



모든 재귀 함수를 반복 함수로 변환할 수 있는가?

모든 반복 함수를 재귀 함수로 변환할 수 있는가?

리스트의 합: 반복 함수 (loop)

```
def sum1(n, arr):  
    sum = 0  
    for i in range(n):  
        sum += arr[i]  
    return sum
```

리스트의 합: 재귀 함수 (recursion)

```
def sum2(n, arr):  
    if n == 0: # 종료 조건  
        return 0  
    else: # 재귀 조건  
        return arr[0] + sum2(n - 1, arr[1:])
```



Tower of Hanoi: 다음 재귀 알고리즘을 반복문으로 구현하시오.

```
def hanoi(n, src, via, dst):  
    if n == 1:  
        print(f"Move from {src} to {dst}")  
    else:  
        hanoi(n - 1, src, dst, via)  
        hanoi(1, src, via, dst)  
        hanoi(n - 1, via, src, dst)
```



Matrix Multiplication: 다음 반복 알고리즘을 재귀 함수로 구현하시오.

```
# N*K 행렬과 K*M 행렬의 곱을 반환한다.  
def mmult(A, B):  
    assert(len(A[0]) == len(B))  
    N, K, M = len(A), len(A[0]), len(B[0])  
    C = [[0] * M for _ in range(N)]  
    for i in range(N):  
        for j in range(M):  
            for k in range(K):  
                C[i][j] += A[i][k] * B[k][j]  
    return C
```



순열과 조합

완전 탐색 문제는 순열과 조합을 통해 해결할 수 있는 경우가 많다.

이때 파이썬의 itertools 모듈에 있는 순열과 조합 관련 라이브러리를 활용할 수 있다.

순열: 집합의 원소들을 순서를 고려하여 나열하는 방법

조합: 집합의 원소들을 순서를 고려하지 않고 나열하는 방법

중복 순열: 원소들의 중복을 허용하는 순열

중복 조합: 원소들의 중복을 허용하는 조합



Lec. 02. 무식하게 풀기

```
import itertools as itt

print("permutations:") # 순열
for perm in itt.permutations(range(4), 2):
    print(perm)

print("product:") # 중복 순열
for prod in itt.product(range(4), repeat = 2):
    print(prod)

print("combinations:") # 조합
for comb in itt.combinations(range(4), 2):
    print(comb)

print("combinations_with_replacement:") # 중복 조합
for comb in itt.combinations_with_replacement(range(4), 2):
    print(comb)
```





Lec. 02. 무식하게 풀기

permutations:

(0, 1)
(0, 2)
(0, 3)
(1, 0)
(1, 2)
(1, 3)
(2, 0)
(2, 1)
(2, 3)
(3, 0)
(3, 1)
(3, 2)

product:

(0, 0)
(0, 1)
(0, 2)
(0, 3)
(1, 0)
(1, 1)
(1, 2)
(1, 3)
(2, 0)
(2, 1)
(2, 2)
(2, 3)
(3, 0)
(3, 1)
(3, 2)
(3, 3)

combinations:

(0, 1)
(0, 2)
(0, 3)
(1, 2)
(1, 3)
(2, 3)

combinations_with_replacement:

(0, 0)
(0, 1)
(0, 2)
(0, 3)
(1, 1)
(1, 2)
(1, 3)
(2, 2)
(2, 3)
(3, 3)

permutations_with_replacement = Cartesian product

$$\{1,2\} \times \{1,2\} = \{(1,1), (1,2), (2,1), (2,2)\}$$



완전 탐색 레시피

1. 가능한 모든 답의 후보를 만드는 과정을 여러 개의 선택으로 나눈다.
각 선택은 답의 후보를 만드는 과정의 한 조각이 된다.
2. 그 중 하나의 조각을 선택해 답의 일부를 만들고,
나머지 답을 재귀 호출을 통해 완성한다.
3. 조각이 하나밖에 남지 않은 경우, 혹은 하나도 남지 않은 경우
답을 생성했으므로 이것을 기저 사례로 선택해 처리한다.
4. 완전 탐색의 시간 복잡도는 일반적으로 탐색해야 하는 후보의 수에 비례한다.
따라서 탐색 공간의 크기를 계산해서 시간 안에 계산할 수 없다면 다른 설계 패러다임을 적용한다.



AL2.2.1: 시계맞추기 (ID: CLOCKSUNC)

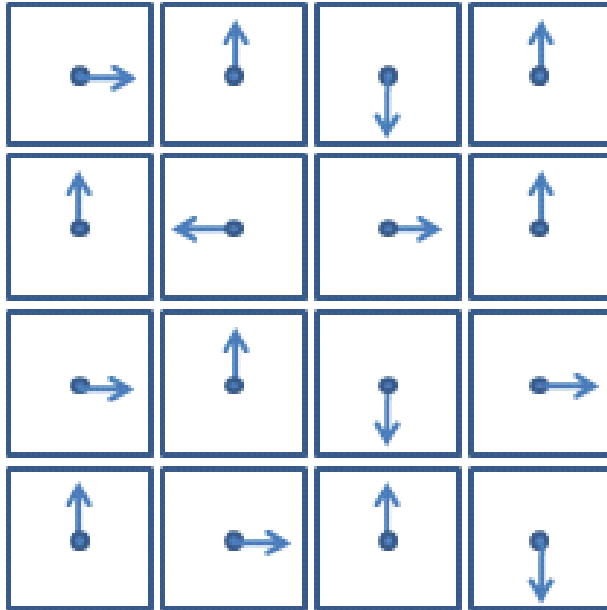
문제 그림과 같이 4×4 개의 격자 형태로 배치된 16개의 시계가 있다. 이 시계들은 모두 12시, 3시, 6시, 혹은 9시를 가리키고 있다. 이 시계들이 모두 12시를 가리키도록 바꾸고 싶다.

시계의 시간을 조작하는 유일한 방법은 모두 10개 있는 스위치들을 조작하는 것으로, 각 스위치들은 모두 적게는 3개에서 많게는 5개의 시계에 연결되어 있다. 한 스위치를 누를 때마다, 해당 스위치와 연결된 시계들의 시간은 3시간씩 앞으로 움직인다. 스위치들과 그들이 연결된 시계들의 목록은 다음과 같다.

시계들은 맨 윗줄부터, 왼쪽에서 오른쪽으로 순서대로 번호가 매겨졌다고 가정하자. 시계들이 현재 가리키는 시간들이 주어졌을 때, 모든 시계를 12시로 돌리기 위해 최소한 눌러야 할 스위치의 수를 계산하는 프로그램을 작성하시오.



Lec. 02. 무식하게 풀기



스위치 번호	연결된 시계들
0	0, 1, 2
1	3, 7, 9, 11
2	4, 10, 14, 15
3	0, 4, 5, 6, 7
4	6, 7, 8, 10, 12
5	0, 2, 14, 15
6	3, 14, 15
7	4, 5, 7, 14, 15
8	1, 2, 3, 4, 5
9	3, 4, 5, 9, 13



입력

첫 줄에 테스트 케이스의 개수 C (≤ 30) 가 주어진다.

각 테스트 케이스는 한 줄에 16개의 정수로 주어지며, 각 정수는 0번부터 15번까지 각 시계가 가리키고 있는 시간을 12, 3, 6, 9 중 하나로 표현한다.

출력

각 테스트 케이스당 한 줄을 출력한다. 시계들을 모두 12시로 돌려놓기 위해 눌러야 할 스위치의 최소 수를 출력한다. 만약 이것이 불가능할 경우 -1 을 출력한다.



Lec. 02. 무식하게 풀기

예제 입력

```
2
12 6 6 6 6 6 12 12 12 12 12 12 12 12 12 12
12 9 3 12 6 6 9 3 12 9 12 9 12 12 6 6
```

예제 출력

```
2
9
```



무식하게 풀 수 있을까?

스위치를 누르는 순서는 전혀 중요하지 않다.

우리가 계산해야 할 것은 각 스위치를 몇 번이나 누를 것이냐 뿐이다.

어떤 스위치를 네 번 누르면 연결된 시계는 모두 12시간씩 앞으로 이동한다.

12시간이 지나면 제 자리이므로, 어떤 스위치건 간에 최대 세 번만 눌러보면 된다.

각 스위치를 누르는 횟수: 0에서 3사이의 정수

스위치의 개수는 10개이므로, 전체 경우의 수는 $4^{10} = 1,048,576$

불가능할 경우 -1을 출력해야 하므로, 완전 탐색을 적용해 본다.



Lec. 02. 무식하게 풀기

...

P.2.1: 시계 맞추기 (ID: CLOCKSUNC): (1) 반복적 완전 탐색

...

`INF = 31` # 스위치의 개수(10) * 최대 횟수(3) 보다 클 수 없다.

`for _ in range(int(input())):`

시계가 가리키고 있는 시간: 12, 3, 6, 9를 0, 1, 2, 3으로 변환

`clocks = [(int(x) // 3) % 4 for x in input().split()]`

`mincnt = clocksync(clocks)` # 스위치를 눌러야 할 최소 횟수

`print(mincnt if mincnt < INF else -1)`

→ 편의를 위해 시계의 정보를 0, 1, 2, 3 으로 변환

12시: 0, 3시: 1, 6시: 2, 9시: 3

스위치를 누르면 연결된 시계의 값이 1 증가 ($\text{mod } 4$)



Lec. 02. 무식하게 풀기

linked = { # 각 스위치별로 연결된 시계의 번호를 저장

0: [0, 1, 2],

1: [3, 7, 9, 11],

2: [4, 10, 14, 15],

3: [0, 4, 5, 6, 7],

4: [6, 7, 8, 10, 12],

5: [0, 2, 14, 15],

6: [3, 14, 15],

7: [4, 5, 7, 14, 15],

8: [1, 2, 3, 4, 5],

9: [3, 4, 5, 9, 13]

}

딕셔너리로 저장:

(key, value) = (스위치 번호, 연결된 시계 리스트)



Lec. 02. 무식하게 풀기

모든 시계가 12시를 가리키면 True, 아니면 False

```
def aligned(clocks):
```

```
    return not any(clocks)
```

하나라도 0(12시)이 아닌 시계가 있으면 False

switch번 스위치와 연결된 시계의 시간을 3시간 앞으로 이동

```
def push(clocks, switch):
```

```
    for clock in linked[switch]:
```

```
        clocks[clock] = (clocks[clock] + 1) % 4
```

```
return ret
```



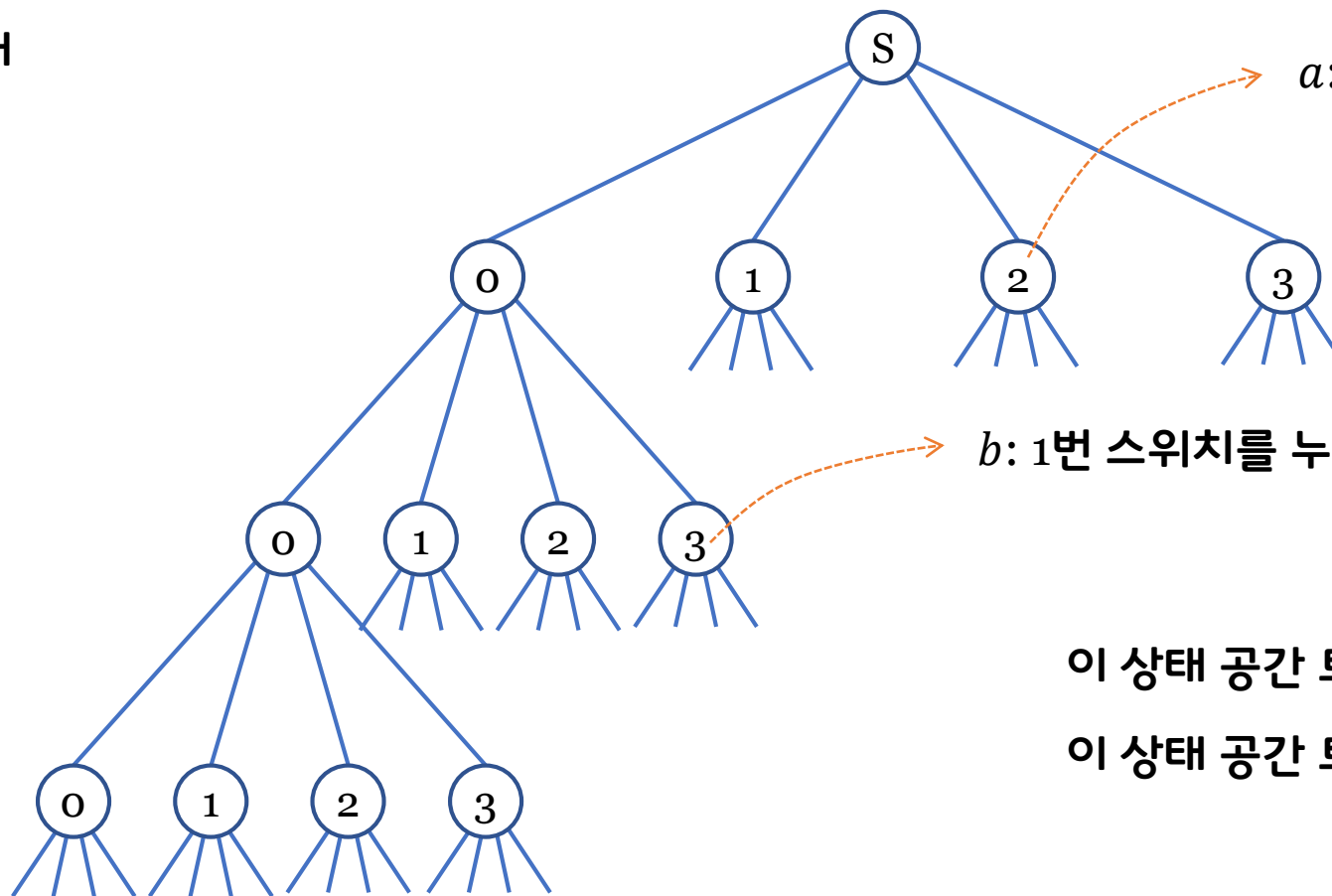
완전 탐색의 상태 공간 트리(SST, *State Space Tree*)

S: 초기상태

0번 스위치

1번 스위치

2번 스위치



a : 0번 스위치를 누른 횟수

b : 1번 스위치를 누른 횟수

이 상태 공간 트리의 노드 개수는?

이 상태 공간 트리의 최대 깊이는?



Lec. 02. 무식하게 풀기

시계 맞추기 문제를 재귀적 완전탐색(DFS)으로 해결

```
def clocksync(clocks, switch):
```

```
    if switch == 10: # 모든 스위치를 눌렀으므로
        # 모든 스위치가 12시면 0, 아니면 INF 반환
```

```
        return 0 if aligned(clocks) else INF
```

```
    else:
```

```
        ret = INF
```

```
        for cnt in range(4):
```

```
            # 현재 switch를 누른 횟수 + 다음 스위치의 횟수로 재귀
```

```
            ret = min(ret, cnt + clocksync(clocks, switch + 1)) # 최솟값 업데이트
```

```
            push(clocks, switch) # switch번 스위치를 한 번 누름
```

```
        return ret
```

첫 호출시에 0번 스위치부터 시작

```
mincnt = clocksync(clocks, 0)
```



Lec. 02. 무식하게 풀기

switch번 스위치와 연결된 시계의 시간을 cnt만큼 앞으로 이동

```
def push(clocks, switch, cnt):  
    for clock in linked[switch]:  
        clocks[clock] = (clocks[clock] + cnt) % 4
```

시계 맞추기 문제를 중복 순열을 이용하여 해결

```
from itertools import product  
def clocksync(clocks):  
    ret = INF  
    for prod in product(range(4), repeat = 10):  
        pushed = clocks[:] # 원래 스위치 정보로 시작하여  
        for switch in range(10): # 모든 스위치를 각 횟수만큼 눌러본다.  
            push(pushed, switch, prod[switch])  
        if aligned(pushed):  
            ret = min(ret, sum(prod))  
    return ret
```





발상의 전환: 스위치에 연결된 시계로부터 시계에 연결된 스위치로!

```
D = {i:[] for i in range(16)}  
for i in range(10):  
    for j in linked[i]:  
        D[j].append(i)  
for key in D:  
    print(key, D[key], sep=": ")
```

```
0: [0, 3, 5]  
1: [0, 8]  
2: [0, 5, 8]  
3: [1, 6, 8, 9]  
4: [2, 3, 7, 8, 9]  
5: [3, 7, 8, 9]  
6: [3, 4]  
7: [1, 3, 4, 7]  
8: [4]  
9: [1, 9]  
10: [2, 4]  
11: [1]  
12: [4]  
13: [9]  
14: [2, 5, 6, 7]  
15: [2, 5, 6, 7]
```



Lec. 02. 무식하게 풀기

```

0: [0, 3, 5]
1: [0, 8]
2: [0, 5, 8]
3: [1, 6, 8, 9]
4: [2, 3, 7, 8, 9]
5: [3, 7, 8, 9]
6: [3, 4]
7: [1, 3, 4, 7]
8: [4]
9: [1, 9]
10: [2, 4]
11: [1]
12: [4]
13: [9]
14: [2, 5, 6, 7]
15: [2, 5, 6, 7]

```

8번 시계는 4번 스위치에만 연결되어 있다.

8번 시계가 12시가 되려면?

4번 스위치를 누르는 횟수는 8번 시계에 의해 결정될 수 있다.

11번 시계도 1번 스위치에만 연결되어 있다.

1번 스위치를 누르는 횟수는 11번 시계에 의해 결정적이다.

13번 시계도 9번 스위치에만 연결되어 있다.

9번 스위치를 누르는 횟수는 13번 시계에 의해 결정적이다.

4번 스위치를 누르는 횟수 = $(4 - \text{clocks}[8]) \% 4$

1번 스위치를 누르는 횟수 = $(4 - \text{clocks}[11]) \% 4$

9번 스위치를 누르는 횟수 = $(4 - \text{clocks}[13]) \% 4$



Lec. 02. 무식하게 풀기

1, 4, 9번 스위치: 11, 8, 13번 시계에 의해 결정적이다.

```
D = {i: (4 - clocks[j]) % 4 for i, j in zip([1, 4, 9], [11, 8, 13])}
```

```
for sw in D: # 누르는 횟수가 결정적인 스위치에 대해서
```

```
    for _ in range(D[sw]): # 해당 스위치를 해당 횟수만큼 누른다.
```

```
        push(clocks, sw)
```

1, 4, 9번 스위치가 결정적이므로, 완전 탐색의 탐색 공간이 확연히 줄어든다.

전체 경우의 수는 $4^7 = 16,384$ (cf. $4^{10} = 1,048,576$)



그렇다면, 다른 스위치도 결정적일 수 있을까?

0: [0, 3, 5]
1: [0, 8]
2: [0, 5, 8]
3: [1, 6, 8, 9]
4: [2, 3, 7, 8, 9]
5: [3, 7, 8, 9]
6: [3, 4]
7: [1, 3, 4, 7]
8: [4]
9: [1, 9]
10: [2, 4]
11: [1]
12: [4]
13: [9]
14: [2, 5, 6, 7]
15: [2, 5, 6, 7]

8번 시계는 4번 스위치에 연결되어 있다.

4번 스위치의 횃수를 결정할 수 있다.

11번 시계는 1번 스위치에 연결되어 있다.

1번 스위치의 횃수를 결정할 수 있다.

13번 시계는 9번 스위치에 연결되어 있다.

9번 스위치의 횃수를 결정할 수 있다.

12번 시계는 4번 스위치에 연결되어 있다.

8번 시계에 의해 4번 스위치의 횃수는 결정되어 있으므로,
고려할 필요가 없다.



Lec. 02. 무식하게 풀기

```

0: [0, 3, 5]
1: [0, 8]
2: [0, 5, 8]
3: [1, 6, 8, 9]
4: [2, 3, 7, 8, 9]
5: [3, 7, 8, 9]
6: [3, 4]
7: [1, 3, 4, 7]
8: [4]
9: [1, 9]
10: [2, 4]
11: [1]
12: [4]
13: [9]
14: [2, 5, 6, 7]
15: [2, 5, 6, 7]

```

누르는 횟수가 이미 결정되어 있는 스위치: 1, 4, 9

6번 시계는 3, 4번 스위치에 연결되어 있다.

4번 스위치를 정해진 횟수만큼 누른 후,

3번 스위치를 누르는 횟수를 결정할 수 있다.

10번 시계는 2, 4번 스위치에 연결되어 있다.

4번 스위치를 누른 후, 2번 스위치의 횟수를 결정할 수 있다.

9번 시계는 1, 9번 스위치에 연결되어 있다.

1, 9 번 스위치는 누르는 횟수가 이미 결정되어 있는 상태다.



Lec. 02. 무식하게 풀기

0: [0, 3, 5]
1: [0, 8]
2: [0, 5, 8]
3: [1, 6, 8, 9]
4: [2, 3, 7, 8, 9]
5: [3, 7, 8, 9]
6: [3, 4]
7: [1, 3, 4, 7]
8: [4]
9: [1, 9]
10: [2, 4]
11: [1]
12: [4]
13: [9]
14: [2, 5, 6, 7]
15: [2, 5, 6, 7]

누르는 횟수가 이미 결정되어 있는 스위치: 1, 2, 3, 4, 9

7번 시계는 1, 3, 4, 7번 스위치에 연결되어 있다.

1, 3, 4번 스위치가 결정되어 있으므로, 7번 스위치를 결정할 수 있다.



Lec. 02. 무식하게 풀기

0: [0, 3, 5]
1: [0, 8]
2: [0, 5, 8]
3: [1, 6, 8, 9]
4: [2, 3, 7, 8, 9]
5: [3, 7, 8, 9]
6: [3, 4]
7: [1, 3, 4, 7]
8: [4]
9: [1, 9]
10: [2, 4]
11: [1]
12: [4]
13: [9]
14: [2, 5, 6, 7]
15: [2, 5, 6, 7]

누르는 횟수가 이미 결정되어 있는 스위치: 1, 2, 3, 4, 7, 9

5번 시계는 3, 7, 8, 9번 스위치에 연결되어 있다.

8번 스위치를 결정할 수 있다.

4번 시계는 2, 3, 7, 8, 9번 스위치에 연결되어 있다.

위에서 8번 스위치를 결정했으므로, 고려할 필요가 없다.



Lec. 02. 무식하게 풀기

0: [0, 3, 5]
1: [0, 8]
2: [0, 5, 8]
3: [1, 6, 8, 9]
4: [2, 3, 7, 8, 9]
5: [3, 7, 8, 9]
6: [3, 4]
7: [1, 3, 4, 7]
8: [4]
9: [1, 9]
10: [2, 4]
11: [1]
12: [4]
13: [9]
14: [2, 5, 6, 7]
15: [2, 5, 6, 7]

누르는 횟수가 이미 결정되어 있는 스위치: 1, 2, 3, 4, 7, 8, 9

1번 시계는 0, 8번 스위치에 연결되어 있다.

0번 스위치를 결정할 수 있다.

3번 시계는 1, 6, 8, 9번 스위치에 연결되어 있다.

6번 스위치를 결정할 수 있다.



Lec. 02. 무식하게 풀기

0: [0, 3, 5]
1: [0, 8]
2: [0, 5, 8]
3: [1, 6, 8, 9]
4: [2, 3, 7, 8, 9]
5: [3, 7, 8, 9]
6: [3, 4]
7: [1, 3, 4, 7]
8: [4]
9: [1, 9]
10: [2, 4]
11: [1]
12: [4]
13: [9]
14: [2, 5, 6, 7]
15: [2, 5, 6, 7]

누르는 횟수가 이미 결정되어 있는 스위치: 0, 1, 2, 3, 4, 6, 7, 8, 9

0번 시계는 0, 3, 5번 스위치에 연결되어 있다.

5번 스위치를 결정할 수 있다.

모든 스위치가 결정되었으므로, 나머지는 고려할 필요가 없다.



Lec. 02. 무식하게 풀기

주어진 스위치-시계 연결 구조에서 **모든 스위치가 결정적**이다!

주어진 시계 정보에서 이미 모든 스위치의 횃수가 결정되어 있으므로,
각 스위치를 정해진 횃수만큼 눌러보고, 모든 시계가 12시인지 확인한다.

이러한 결정 순서는 스위치 간의 의존 관계를 방향 그래프로 나타냈을 때 위상 정렬 순서로 볼 수 있다.

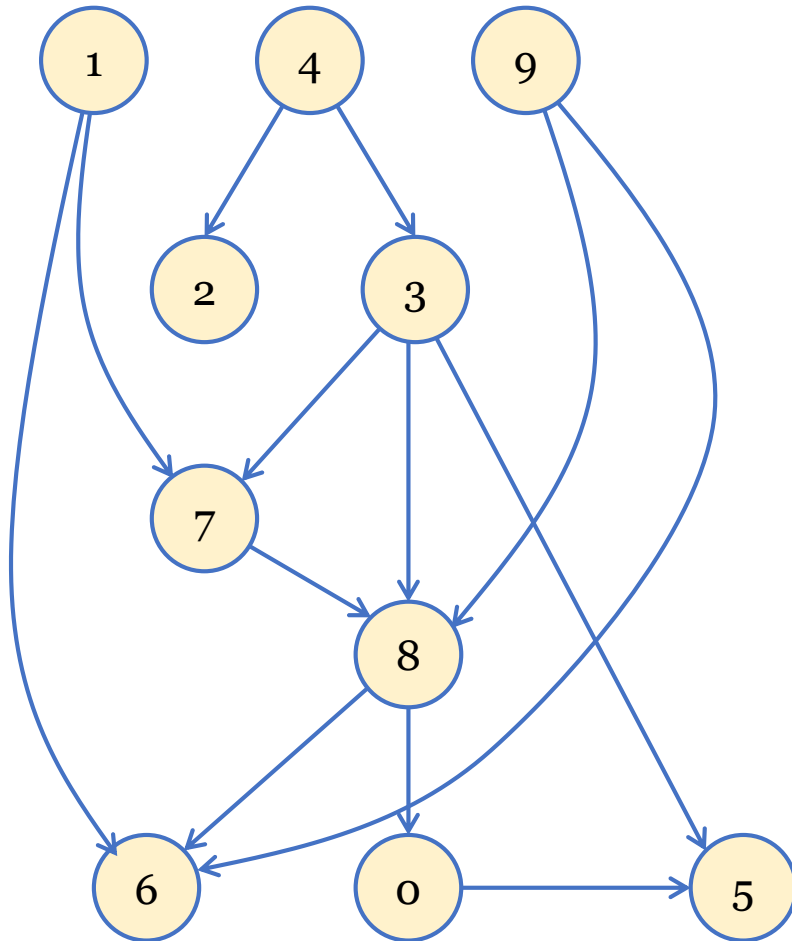
따라서 스위치를 눌러보는 순서는

스위치 간의 **의존 그래프**에서 **위상 정렬**(*topological sort*)된 순서를 따르면 된다.



Lec. 02. 무식하게 풀기

각 스위치의 의존 그래프: *dependency graph*



```

sequence = [
    (8, linked[4]), # 8번 시계, 4번 스위치
    (11, linked[1]), # 11번 시계, 1번 스위치
    (13, linked[9]), # 13번 시계, 9번 스위치
    (6, linked[3]), # 6번 시계, 3번 스위치
    (10, linked[2]), # 10번 시계, 2번 스위치
    (7, linked[7]), # 7번 시계, 7번 스위치
    (4, linked[8]), # 4번 시계, 8번 스위치
    (1, linked[0]), # 1번 시계, 0번 스위치
    (3, linked[6]), # 3번 시계, 6번 스위치
    (0, linked[5]), # 0번 시계, 5번 스위치
]
  
```



발상의 전환으로 시계 맞추기 문제 해결

```
def clocksync(clocks):  
    ret = 0  
    for clock, linked_clock in sequence:  
        cnt = (4 - clocks[clock]) % 4 # clock번 시계를 12시로 만드는 횟수  
        for other in linked_clock: # 해당하는 스위치와 연결된 시계를 3시간 이동  
            clocks[other] = (clocks[other] + cnt) % 4  
        ret += cnt  
    return -1 if any(clocks) else ret
```

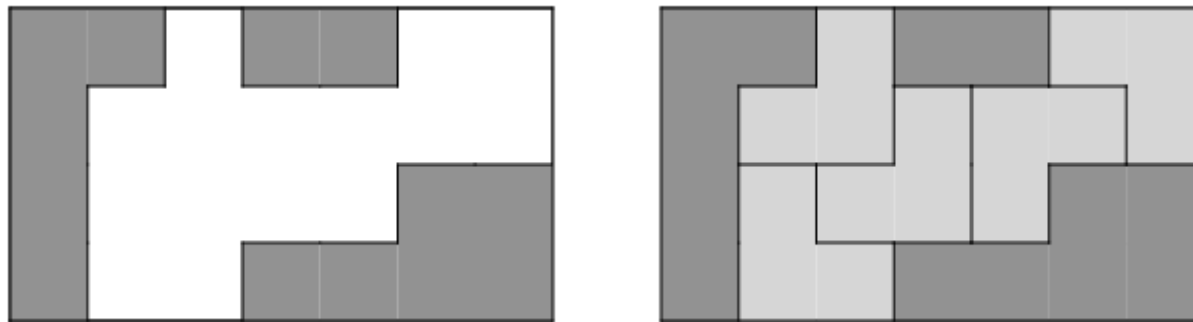


Lec. 02. 무식하게 풀기

AL2.2.2: 게임판 덮기 (ID: BOARDCOVER)

문제 $H \times W$ 크기의 게임판이 있습니다. 게임판은 검은 칸과 흰 칸으로 구성된 격자 모양을 하고 있는데 이 중 모든 흰 칸을 3칸짜리 L자 모양의 블록으로 덮고 싶습니다. 이 때 블록들은 자유롭게 회전해서 놓을 수 있지만, 서로 겹치거나, 검은 칸을 덮거나, 게임판 밖으로 나가서는 안 됩니다. 아래 그림은 한 게임판과 이를 덮는 방법을 보여줍니다.

게임판이 주어질 때 이를 덮는 방법의 수를 계산하는 프로그램을 작성하세요.





입력

입력의 첫 줄에는 테스트 케이스의 수 C ($C \leq 30$) 가 주어집니다. 각 테스트 케이스의 첫 줄에는 2개의 정수 H, W ($1 \leq H, W \leq 20$) 가 주어집니다. 다음 H 줄에 각 W 글자로 게임판의 모양이 주어집니다. # 은 검은 칸, . 는 흰 칸을 나타냅니다. 입력에 주어지는 게임판에 있는 흰 칸의 수는 50 을 넘지 않습니다.

출력

한 줄에 하나씩 흰 칸을 모두 덮는 방법의 수를 출력합니다.



Lec. 02. 무식하게 풀기

예제 입력

```
3
3 7
#.....#
#.....#
##...##
3 7
#.....#
#.....#
##..###
8 10
#####
#.....#
#.....#
#.....#
#.....#
#.....#
#.....#
#####
```

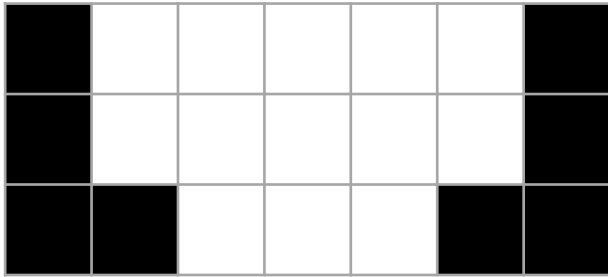
예제 출력

```
0
2
1514
```

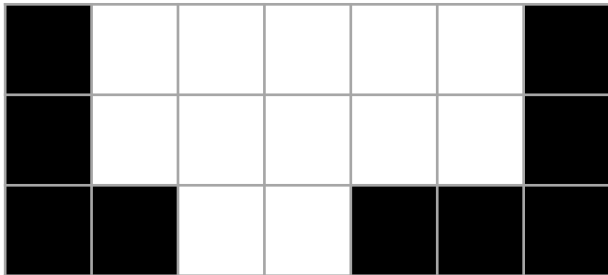


Lec. 02. 무식하게 풀기

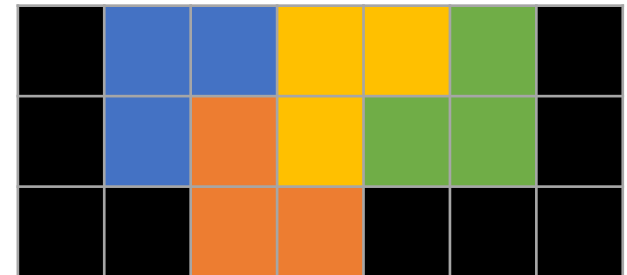
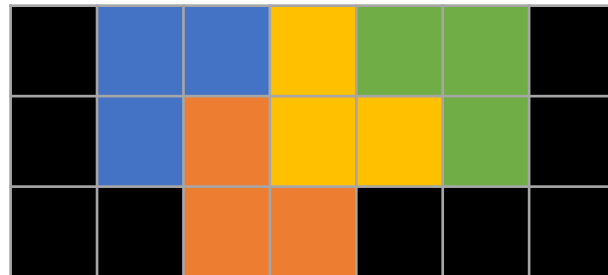
무식하게 풀 수 있을까?



흰 칸이 모두 13칸이므로 L 모양 블록으로 다 덮을 수 없다.



흰 칸이 12칸이므로 두 가지 방법으로 다 덮을 수 있다.





무식하게 풀어도 될까?

흰 칸의 개수가 50을 넘지 않으므로 최대 $\left\lfloor \frac{50}{3} \right\rfloor = 16$ 개의 블록을 놓을 수 있다.

한 블록을 놓을 수 있는 경우의 수는 네 가지이므로, 가능한 답의 상한은 $4^{16} = 2^{32}$ 개가 된다.

무식하게 풀면 시간 제한에 걸리지 않을까?

두 번째 테스트 케이스: 흰 칸의 수가 12개, 경우의 수는 2개 (탐색 공간: $4^4 = 2^8 = 512$ 개)

세 번째 테스트 케이스: 흰 칸의 수가 48개, 경우의 수는 1514개 (탐색 공간: $4^{16} = 2^{32}$ 개)

이유: 각 단계에서 실제 선택할 수 있는 블록 배치는 4가지가 아니라 크게 제한되기 때문이다.



게임판을 덮을 수 있는 모든 경우를 생성하는 완전 탐색을 이용해 해결

게임판에서 흰 칸의 수가 3의 배수가 아닐 경우: 해답이 없음

이 외의 경우:

문제의 답을 생성하는 과정을 여러 조각으로 나눠 한 조각에 한 블록을 내려놓는다.

재귀 함수:

주어진 게임판에서 블록을 한 개 내려놓고

남은 흰 칸들을 재귀 호출을 이용해 덮도록 한다. (DFS)



Lec. 02. 무식하게 풀기

```
'''
```

P.2.2. 게임판 덮기 (ID: BOARDCOVER)

```
'''
```

```
for _ in range(int(input())):  
    H, W = map(int, input().split()) # 게임판의 크기  
    board = [input() for _ in range(H)] # 게임판의 상태  
    print(boardcover(H, W, board)) # 흰 칸을 덮는 방법의 수
```



Lec. 02. 무식하게 풀기

```

3 7
#.....#
#.....#
##....##

```

흰 칸의 개수: $5 + 5 + 3 = 13$ 개

모두 덮는 방법이 없음

```

3 7
#.....#
#.....#
##..###

```

흰 칸의 개수: $5 + 5 + 2 = 12$ 개

모두 덮는 방법이 여러 개 있을 수 있음

```

board2 =
[[1, 0, 0, 0, 0, 0, 1],
 [1, 0, 0, 0, 0, 0, 1],
 [1, 1, 0, 0, 1, 1, 1]]

```

편리를 위해 #과 .을 0과 1로 바꿈

블록을 놓을 때, 칸의 값을 $+1$

블록을 제거할 때, 칸의 값을 -1



Lec. 02. 무식하게 풀기

```
from itertools import product
```

```
# 게임판 덮기 문제를 완전 탐색으로 해결
```

```
def boardcover(H, W, board):
```

```
    # 편의를 위해 board의 값을 0(.)과 1(#)로 바꾼다.
```

```
    board2 = [[0] * W for _ in range(H)]
```

```
    for i, j in product(range(H), range(W)):
```

```
        board2[i][j] = [0, 1][board[i][j] == '#']
```

```
    # 흰 칸의 개수가 3의 배수가 아니면 모두 덮을 수 없다.
```

```
    if sum([row.count(0) for row in board2]) % 3 != 0:
```

```
        return 0
```

```
    else:
```

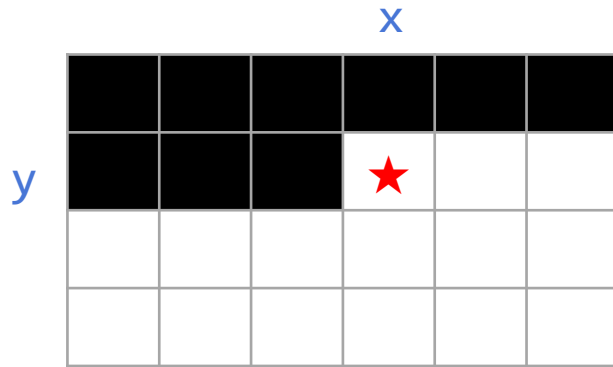
```
        return cover(H, W, board2)
```

. 이면 0, # 이면 1

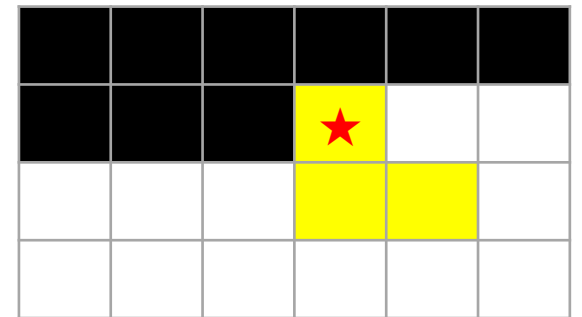
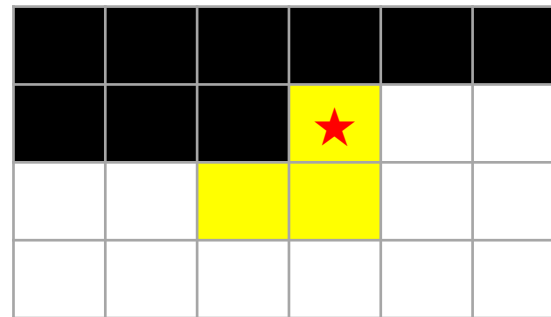
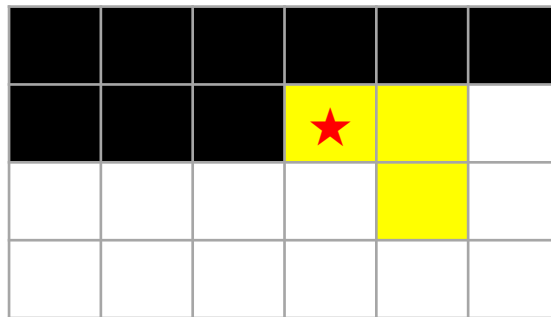
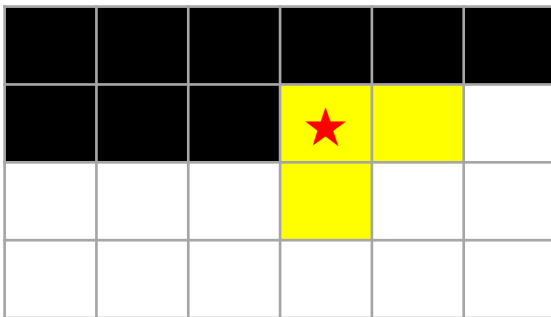


Lec. 02. 무식하게 풀기

같은 배치를 중복으로 세지 않도록 하는 방법은?



특정한 순서로 **하나의 배치**는 **한가지 방법**으로만 생성할 수 있도록 한다.
 비어 있는 칸 중에서 가장 **윗 줄**, 가장 **왼쪽**에 있는 칸을 덮도록 한다.
 왼쪽과 그 위의 칸은 모두 덮여 있으므로, **네 가지**의 경우의 수가 나온다.

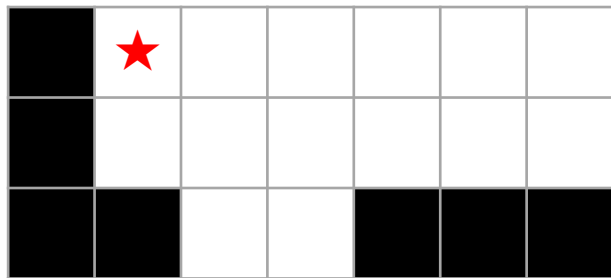




Lec. 02. 무식하게 풀기

아직 채우지 못한 칸 중 가장 윗줄 가장 왼쪽에 있는 칸을 찾는다.

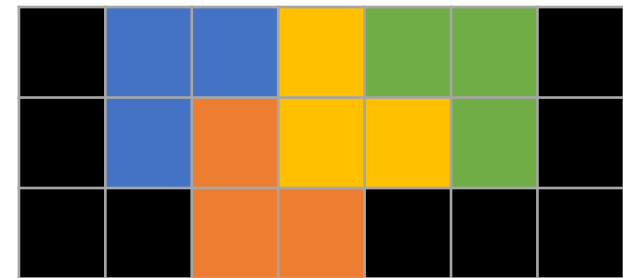
```
def find_white(H, W, board):
    for i in range(H):
        for j in range(W):
            if board[i][j] == 0:
                return i, j
    return -1, -1
```



return 0, 1



return 1, 2



return -1, -1



각 블록의 유형은 상대좌표의 목록으로 저장한다.

주어진 칸을 덮을 수 있는 네 가지 방법

(dy, dx): 블록을 구성하는 세 칸의 상대적 위치

```
coverType = [  
    [(0, 0), (1, 0), (0, 1)],  
    [(0, 0), (0, 1), (1, 1)],  
    [(0, 0), (1, 0), (1, 1)],  
    [(0, 0), (1, 0), (1, -1)],  
]
```





Lec. 02. 무식하게 풀기

(ny, nx) : 상대좌표를 저장한 *coverType*을 이용해서 블록을 놓을 칸을 결정 (3칸)

1. (ny, nx) 가 보드의 범위를 벗어난 경우:

*delta*를 더할 필요가 없으므로, *ok*를 *False*로 두고 계속 진행

2. (ny, nx) 가 보드의 범위를 벗어나지 않은 경우:

*delta*를 더한 후, 해당 칸의 값이 1보다 크면 검은 칸이거나 이미 덮은 칸이므로 *ok*를 *False*로 두고 계속 진행

*ok*가 *False*일 때 바로 리턴하지 않고 계속 진행하는 이유:

블록의 모든 칸을 놓아 두어야, *delta*가 -1일 때 정상적으로 제거할 수 있다.



Lec. 02. 무식하게 풀기

```
# board의 (y, x)를 type 유형으로 처리한다.
# delta가 1이면 덮고, -1이면 덮었던 블록을 제거한다.
# 덮을 수 있으면 True, 덮을 수 없으면 False를 반환한다.
def place(board, y, x, type, delta, H, W):
    ok = True
    for i in range(3):
        ny = y + coverType[type][i][0]
        nx = x + coverType[type][i][1]
        if not ((0 <= ny < H) and (0 <= nx < W)): # 보드 밖으로 나간 경우
            ok = False
        else:
            board[ny][nx] += delta # 칸을 채우거나 제거한다.
            if (board[ny][nx] > 1): # 검은 칸이거나 겹쳐서 덮는 경우
                ok = False
    return ok
```





```
# board의 모든 빈 칸을 덮을 수 있는 방법의 수를 반환
# board[i][j] == 1: 덮인 칸(검은 칸)
# board[i][j] == 0: 빈 칸(흰 칸)
def cover(H, W, board):
    y, x = find_white(H, W, board) # 비어 있는 첫 번째 칸을 찾는다.
    if y == -1:
        return 1 # 모든 칸을 채웠으므로 1을 반환한다.
    else:
        ret = 0
        for type in range(4): # 모든 블록 유형에 대해서 덮어본다.
            if (place(board, y, x, type, 1, H, W)):
                ret += cover(H, W, board) # 만약 덮을 수 있으면 재귀 호출한다.
            place(board, y, x, type, -1, H, W) # 덮은 블록을 제거한다.
        return ret
```

Any Questions?





P.2.3: 보글 게임 (ID: BOGGLE)

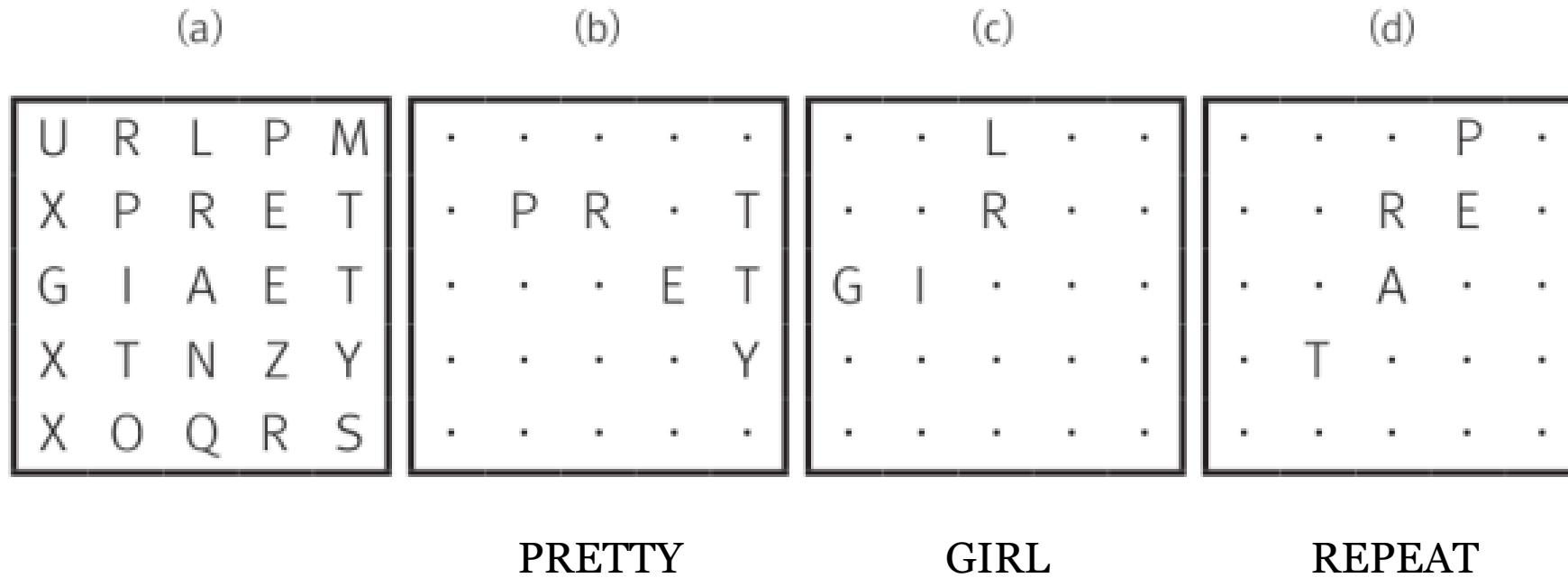
문제 보글(Boggle) 게임은 그림 (a)와 같은 5x5 크기의 알파벳 격자인 게임판의 한 글자에서 시작해서 펜을 움직이면서 만나는 글자를 그 순서대로 나열하여 만들어지는 영어 단어를 찾아내는 게임입니다. 펜은 상하좌우, 혹은 대각선으로 인접한 칸으로 이동할 수 있으며 글자를 건너뛰 수는 없습니다. 지나간 글자를 다시 지나가는 것은 가능하지만, 펜을 이동하지 않고 같은 글자를 여러번 쓸 수는 없습니다.

예를 들어 그림의 (b), (c), (d)는 각각 (a)의 격자에서 PRETTY, GIRL, REPEAT을 찾아낸 결과를 보여줍니다.

보글 게임판과 알고 있는 단어들의 목록이 주어질 때, 보글 게임판에서 각 단어를 찾을 수 있는지 여부를 출력하는 프로그램을 작성하세요.



Lec. 02. 무식하게 풀기





입력

입력의 첫 줄에는 테스트 케이스의 수 C ($C \leq 50$)가 주어집니다. 각 테스트 케이스의 첫 줄에는 각 5줄에 5글자로 보글 게임판이 주어집니다. 게임판의 모든 칸은 알파벳 대문자입니다.

그 다음 줄에는 우리가 알고 있는 단어의 수 N ($1 \leq N \leq 10$)이 주어집니다. 그 후 N 줄에는 한 줄에 하나씩 우리가 알고 있는 단어들이 주어집니다. 각 단어는 알파벳 대문자 1글자 이상 10글자 이하로 구성됩니다.

출력

각 테스트 케이스마다 N 줄을 출력합니다. 각 줄에는 알고 있는 단어를 입력에 주어진 순서대로 출력하고, 해당 단어를 찾을 수 있을 경우 YES, 아닐 경우 NO를 출력합니다.



Lec. 02. 무식하게 풀기

예제 입력

1

URLPM

XPRET

GIAET

XTNZY

XOQRS

6

PRETTY

GIRL

REPEAT

KARA

PANDORA

GIAZAPX

예제 출력

PRETTY YES

GIRL YES

REPEAT YES

KARA NO

PANDORA NO

GIAZAPX YES